

Cuestionario de Java

60 Preguntas con Justificación

Presentado por:

Rafael Ricardez Reyes

Matrícula: 2153223056

Facultad de Ingeniería

Ingeniería en Sistemas Computacionales

20 de mayo de 2026

Cuestionario Justificado

No.	Pregunta	Respuesta	Justificación
1	Polimorfismo dinámico	Implementar métodos del padre con misma firma en hijas.	Cambia comportamiento heredado en tiempo de ejecución.
2	Operadores unarios	Actúan sobre un solo operando (+, -, !, ~).	Solo requieren un valor para operar.
3	Extensión bytecode	.class	Generado por el compilador Java.
4	¿Qué es this?	Apunta al objeto actual.	Permite referirse al objeto dentro de la clase.
5	3 usos de super	Invocar constructor, métodos y variables del padre.	Permite acceder a miembros de la superclase.
6	¿Arreglo tipos?	TRUE	Pueden contener ambos tipos.
7	3 formas crear arreglo	Literal, new T[s], decl. y asignación.	Son las formas válidas de sintaxis en Java.
8	2 tipos polimorfismo	Estático (sobrecarga) y Dinámico (sobrescritura).	Aplican según el tiempo de enlace.
9	¿Qué es interfaz?	Clase con solo firmas de métodos.	Define un contrato sin implementación.
10	¿Variable estática?	TRUE	Pertenece a la clase, no a la instancia.
11	UML asociación	Línea recta.	Representa la relación entre objetos.
12	¿Sobrescribir privados?	No.	No son heredados por las subclases.
13	¿Try requiere catch?	TRUE.	Es parte de la estructura de manejo de errores.
14	¿Qué es Stack?	Almacena variables locales y llamadas.	Gestión de memoria de ejecución.
15	¿Recolector basura?	Libera memoria de objetos sin referencia.	Evita fugas de memoria.
16	¿Sobrescribir estáticos?	FALSE.	Están ligados a la clase (enlace estático).
17	4 ventajas POO	Modularidad, escalabilidad, encapsulamiento, reuso.	Pilares del paradigma de programación.
18	UML constantes	Mayúsculas.	Convención para valores invariables.
19	Catch genérico	Error de compilación.	Ocultaría los bloques de excepciones específicas.
20	¿Métodos estáticos?	Acceso sin instanciar.	No requieren un objeto para ser llamados.
21	Encapsulación	Ocultar datos.	Protege la integridad de la información.
22	Clase concreta	Debe implementar todos los abstractos.	Garantiza la completitud del contrato.
23	3 métodos Object	toString(), equals(), hashCode().	Son métodos raíz de toda clase en Java.
24	Super() primera línea	TRUE.	Requiere inicializar la base antes de proceder.

No.	Pregunta	Respuesta	Justificación
25	Getters y setters	Accesores y mutadores.	Permiten el acceso controlado a atributos.
26	Literal String	Se busca en el Pool.	Optimización de memoria en Strings.
27	Tipos enteros	double, long, int, short.	Diferente capacidad de almacenamiento.
28	¿Para qué <code>this</code> ?	Diferenciar ámbito local vs instancia.	Evita ambigüedad en nombres.
29	Análisis código	Detectar violaciones.	Mejora la calidad y estilo del código.
30	JDK	Compilador + JRE.	Kit necesario para el desarrollo completo.
31	Clases Wrapper	Primitivos como objetos.	Permite uso en colecciones y Genéricos.
32	Overflow short	32767 a -32768.	Comportamiento cíclico en enteros.
33	Elegible GC	Sin referencias activas.	La JVM detecta cuando ya no es alcanzable.
34	¿Referencia múltiples?	FALSE.	Una variable referencia a un solo objeto.
35	Interfaz métodos	public y abstract.	Es su estado por defecto.
36	Anotación	Metadatos del código.	Informan al compilador o framework.
37	Excepción no verificada	Runtime.	No requieren ser declaradas obligatoriamente.
38	Imports	*, clase, o nombre completo.	Permiten usar clases fuera del paquete.
39	Conversión	Implícita y explícita.	Ajuste de tipos de datos.
40	Abstracta sin abstractos	TRUE.	Define una clase que no debe instanciarse.
41	Primitivo vs Referencia	Valor vs Dirección.	Naturaleza distinta de almacenamiento.
42	Hija hereda	Todo excepto privados.	Reglas de visibilidad en herencia.
43	4 tipos herencia	Simple, multi, jerárquica, múltiple.	Tipos de estructuras de clases.
44	Parámetros	Actuales y formales.	Diferencia entre el envío y la definición.
45	Simular herencia múltiple	Mediante interfaces.	Solución al problema de colisión.
46	4 modificadores	private, default, protected, public.	Control de encapsulamiento.
47	Intérprete	Ejecuta bytecode.	Motor de la JVM.
48	Biblioteca estándar	Clases comunes.	Facilita el desarrollo.
49	Final en blanco	En el constructor.	Obligatorio antes de usarla.
50	Objeto elegible	Sin referencia.	JVM lo elimina automáticamente.
51	Heap	Memoria dinámica.	Donde se almacenan los objetos.

No.	Pregunta	Respuesta	Justificación
52	JDK definición	Herramientas + JRE.	Kit de desarrollo completo.
53	String creación	Literal y <code>new</code> .	Métodos válidos de instancia- ción.
54	Paso referencia	Apuntan al mismo objeto.	Los cambios afectan al original.
55	Paso valor	Copia del valor.	No afecta variables externas.
56	Main args	Por línea de comandos.	Permite pasar datos al progra- ma.
57	Identificador	Nombre único.	Identifica elementos del lengua- je.
58	Multiplataforma	TRUE.	Java corre en cualquier SO con JRE.
59	String primitivo	FALSE.	String es una clase.
60	Solo una anotación	FALSE.	Se pueden apilar anotaciones.